

# Contents

|                                                                               |          |
|-------------------------------------------------------------------------------|----------|
| <b>Fortify Platform: Technical Architecture &amp; Implementation Strategy</b> | <b>1</b> |
| Executive Summary . . . . .                                                   | 1        |
| Table of Contents . . . . .                                                   | 1        |
| 1. Technology Stack Comparison . . . . .                                      | 2        |
| 2. Recommended Architecture . . . . .                                         | 3        |
| 3. Database Design (Supabase/PostgreSQL) . . . . .                            | 4        |
| 4. Frontend Implementation (WeWeb) . . . . .                                  | 10       |
| 5. AI Contract Processing Pipeline . . . . .                                  | 15       |
| 6. Authentication & Security . . . . .                                        | 19       |
| 7. Automation Features . . . . .                                              | 21       |
| 8. MVP Timeline with Claude Code . . . . .                                    | 24       |
| 9. Cost Analysis . . . . .                                                    | 27       |
| 10. Implementation Roadmap . . . . .                                          | 29       |
| 11. Next Steps . . . . .                                                      | 31       |
| Conclusion . . . . .                                                          | 32       |
| Appendix . . . . .                                                            | 33       |

## Fortify Platform: Technical Architecture & Implementation Strategy

### Executive Summary

#### Recommended Stack: WeWeb + Supabase + Claude Code

This document provides a comprehensive technical architecture for the Fortify vendor management platform, designed to deliver a rapid MVP (4-5 weeks) while maintaining enterprise-grade scalability for school districts paying \$100k+ annually.

**Key Decision: WeWeb over Retool** - 84% lower monthly costs at scale (\$199 vs \$1,200/month)

- Superior white-label capabilities for district branding - True code ownership and export flexibility
- Better performance for end-user facing dashboards

**Claude Code Impact:** - Reduces development time from 12+ weeks to 4-5 weeks - Eliminates need for full-time developers initially - Enables rapid iteration based on pilot feedback - Development cost: ~\$100 in API credits vs \$72k+ traditional dev

---

### Table of Contents

1. Technology Stack Comparison
2. Recommended Architecture
3. Database Design (Supabase/PostgreSQL)
4. Frontend Implementation (WeWeb)
5. AI Contract Processing Pipeline
6. Authentication & Security
7. Automation Features
8. MVP Timeline with Claude Code

- 9. Cost Analysis
  - 10. Implementation Roadmap
  - 11. Next Steps
- 

## 1. Technology Stack Comparison

### WeWeb vs Retool: Detailed Analysis

**WeWeb Advantages**    **1. True White-Label Control** - Complete CSS/styling freedom (unlike Retool's constraints) - Custom domains per district (orangeusd.fortifyplatform.com) - No "built with Retool" branding limitations - Better for public-facing portals where brand matters

**2. Superior Performance** - Builds actual Vue.js apps (real code, not iframe widgets) - Faster page loads for end users - Better mobile experience out of the box - PWA capabilities for offline access

**3. Cost Structure** - WeWeb: \$79-199/month for unlimited users - Retool: \$50/user/month (gets expensive fast) - At 20 district admins: WeWeb \$199 vs Retool \$1,000/month - **Savings: \$9,612 annually at scale**

**4. Flexibility for Future** - Can export the code if you ever need to - Easier to customize edge cases - Better for client-facing portals vs internal tools - No vendor lock-in concerns

**Retool Advantages**    **1. Speed to Complex Dashboards** - Pre-built chart components are exceptional - Drag-drop SQL query builders - Built-in state management for complex workflows - Better for rapid prototyping of analytics views

**2. Enterprise Features (Out of Box)** - Audit logs automatically tracked - Granular permissions system - Better admin controls for your internal team - SAML/SSO easier to implement

**3. Internal Tool Optimization** - If you need a separate Fortify team dashboard, Retool excels here - Better for operational workflows (approvals, staging reviews)

### Recommended Hybrid Approach

#### WeWeb for District-Facing Dashboards

District User Experience:

- login.orangeusd.fortifyplatform.com
- Beautiful, branded executive dashboard
- Mobile-responsive for superintendent iPad use
- Custom color schemes matching district branding

#### Optional Retool for Internal Fortify Tools

Fortify Team Operations:

- admin.fortifyplatform.com (Retool)
- Contract review/approval workflows
- Multi-district oversight dashboard
- AI parsing staging interface

**Why Hybrid Works:** - Districts never see your internal tools (they get WeWeb excellence) - You get Retool's speed for operational workflows - Best of both worlds at reasonable cost

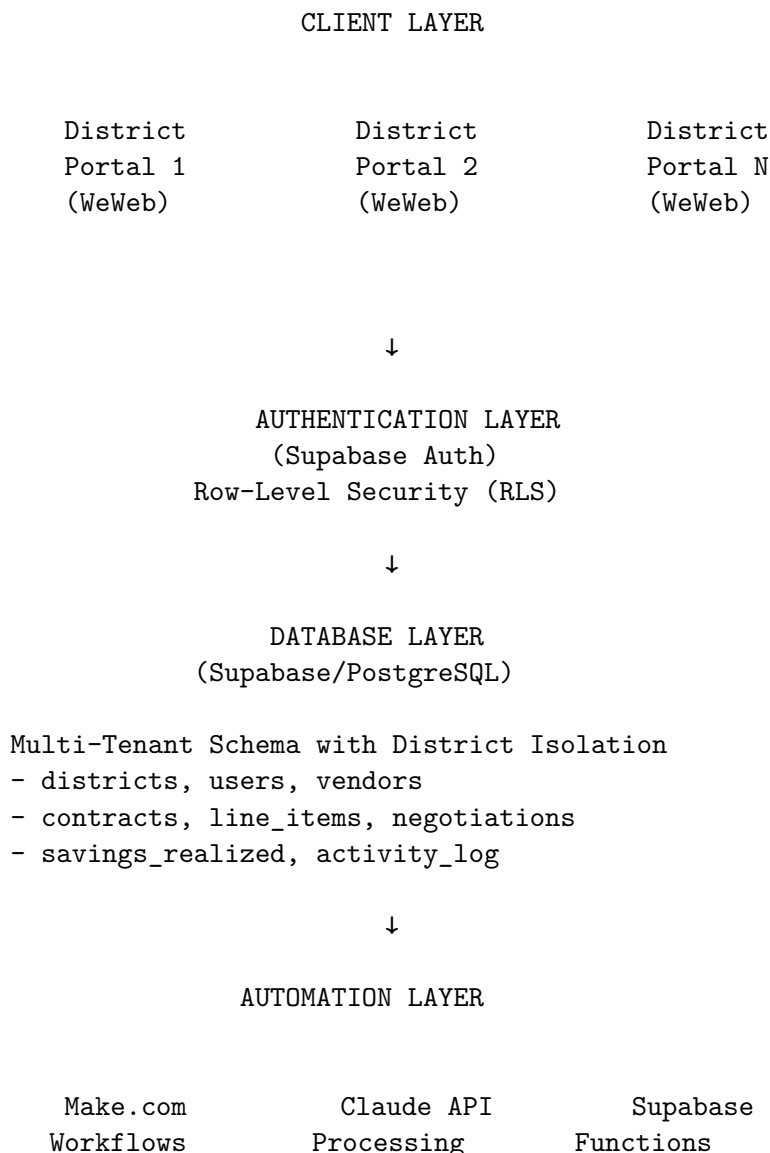
---

## 2. Recommended Architecture

### Core Platform: WeWeb + Supabase (PostgreSQL)

**Why This Stack:** - WeWeb provides beautiful, customizable dashboards with minimal dev time  
- Supabase gives you PostgreSQL with built-in row-level security, auth, and real-time capabilities  
- Both scale seamlessly from 5 districts to 500+ - Combined cost: ~\$300-525/month initially vs \$100k+ annual revenue per client - **Gross margin: 98.7% at scale**

### System Architecture Diagram



- Contract parsing
- Email alerts
- Renewal notifications
- Report generation

### 3. Database Design (Supabase/PostgreSQL)

#### Multi-Tenant Schema with District Isolation

```
-- =====
-- DISTRICTS (Multi-tenant root)
-- =====

CREATE TABLE districts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,
  domain TEXT UNIQUE,
  subscription_tier TEXT CHECK (subscription_tier IN ('pilot', 'standard', 'premium', 'enterprise')),
  enrollment_current INTEGER,
  enrollment_previous INTEGER,
  onboarded_date DATE DEFAULT CURRENT_DATE,
  primary_contact TEXT,
  settings_json JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- USERS (District-scoped with roles)
-- =====

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email TEXT UNIQUE NOT NULL,
  district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
  role TEXT CHECK (role IN ('super_admin', 'district_admin', 'district_editor', 'district_viewer')),
  permissions_json JSONB DEFAULT '{}',
  last_login TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- VENDORS (District-scoped)
-- =====

CREATE TABLE vendors (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
```

```

    vendor_name TEXT NOT NULL,
    category TEXT CHECK (category IN ('software', 'services', 'supplies', 'transportation', 'food')),
    primary_contact TEXT,
    email TEXT,
    phone TEXT,
    website TEXT,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- CONTRACTS (Core entity)
-- =====

CREATE TABLE contracts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
    vendor_id UUID REFERENCES vendors(id) ON DELETE SET NULL,
    contract_name TEXT NOT NULL,
    contract_number TEXT,
    start_date DATE,
    end_date DATE,
    annual_value DECIMAL(12,2),
    renewal_date DATE,
    auto_renew BOOLEAN DEFAULT false,
    status TEXT CHECK (status IN ('active', 'expired', 'pending_renewal', 'under_negotiation', 'cancelled')),
    document_url TEXT,
    parsed_at TIMESTAMPTZ,
    reviewed_by UUID REFERENCES users(id),
    reviewed_at TIMESTAMPTZ,
    ai_confidence_score INTEGER CHECK (ai_confidence_score BETWEEN 0 AND 100),
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- CONTRACT LINE ITEMS (Parsed details)
-- =====

CREATE TABLE contract_line_items (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    contract_id UUID REFERENCES contracts(id) ON DELETE CASCADE,
    description TEXT,
    unit_cost DECIMAL(12,2),
    quantity INTEGER,
    annual_cost DECIMAL(12,2),
    category TEXT,
    parsed_confidence_score INTEGER CHECK (parsed_confidence_score BETWEEN 0 AND 100),

```

```

        created_at TIMESTAMPTZ DEFAULT NOW()
    );

-- =====
-- NEGOTIATIONS (Pipeline tracking)
-- =====

CREATE TABLE negotiations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    contract_id UUID REFERENCES contracts(id) ON DELETE CASCADE,
    district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
    current_annual_spend DECIMAL(12,2),
    proposed_annual_spend DECIMAL(12,2),
    potential_savings DECIMAL(12,2) GENERATED ALWAYS AS (current_annual_spend - proposed_annual_spend) STORED,
    savings_percentage DECIMAL(5,2) GENERATED ALWAYS AS (
        CASE
            WHEN current_annual_spend > 0
            THEN ((current_annual_spend - proposed_annual_spend) / current_annual_spend * 100)
            ELSE 0
        END
    ) STORED,
    status TEXT CHECK (status IN ('identified', 'in_progress', 'vendor_contacted', 'proposal_received')),
    priority TEXT CHECK (priority IN ('low', 'medium', 'high', 'urgent')),
    next_action TEXT,
    next_action_date DATE,
    assigned_to UUID REFERENCES users(id),
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- SAVINGS REALIZED (Success metrics)
-- =====

CREATE TABLE savings_realized (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    negotiation_id UUID REFERENCES negotiations(id) ON DELETE SET NULL,
    contract_id UUID REFERENCES contracts(id) ON DELETE CASCADE,
    district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
    baseline_cost DECIMAL(12,2) NOT NULL,
    new_cost DECIMAL(12,2) NOT NULL,
    savings_amount DECIMAL(12,2) GENERATED ALWAYS AS (baseline_cost - new_cost) STORED,
    savings_percentage DECIMAL(5,2) GENERATED ALWAYS AS (
        CASE
            WHEN baseline_cost > 0
            THEN ((baseline_cost - new_cost) / baseline_cost * 100)
            ELSE 0
        END
    ) STORED,

```

```

realized_date DATE DEFAULT CURRENT_DATE,
validation_status TEXT CHECK (validation_status IN ('pending', 'verified', 'disputed')),
success_fee_owed DECIMAL(12,2),
success_fee_type TEXT CHECK (success_fee_type IN ('savings', 'donation')),
notes TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- ACTIVITY LOG (Audit trail)
-- =====

CREATE TABLE activity_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    district_id UUID REFERENCES districts(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action_type TEXT NOT NULL,
    entity_type TEXT,
    entity_id UUID,
    details_json JSONB DEFAULT '{}',
    timestamp TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- INDEXES for Performance
-- =====

CREATE INDEX idx_contracts_district ON contracts(district_id);
CREATE INDEX idx_contracts_vendor ON contracts(vendor_id);
CREATE INDEX idx_contracts_status ON contracts(status);
CREATE INDEX idx_contracts_renewal_date ON contracts(renewal_date);
CREATE INDEX idx_negotiations_district ON negotiations(district_id);
CREATE INDEX idx_negotiations_status ON negotiations(status);
CREATE INDEX idx_savings_district ON savings_realized(district_id);
CREATE INDEX idx_activity_district ON activity_log(district_id);
CREATE INDEX idx_activity_timestamp ON activity_log(timestamp DESC);

-- =====
-- ROW-LEVEL SECURITY (RLS) Policies
-- =====

-- Enable RLS on all tables
ALTER TABLE districts ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE vendors ENABLE ROW LEVEL SECURITY;
ALTER TABLE contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE contract_line_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE negotiations ENABLE ROW LEVEL SECURITY;
ALTER TABLE savings_realized ENABLE ROW LEVEL SECURITY;
ALTER TABLE activity_log ENABLE ROW LEVEL SECURITY;

```

```

-- Super admin can see everything
CREATE POLICY super_admin_all ON contracts
  FOR ALL USING (
    EXISTS (
      SELECT 1 FROM users
      WHERE users.id = auth.uid()
      AND users.role = 'super_admin'
    )
  );

-- District users can only see their district's data
CREATE POLICY district_isolation ON contracts
  FOR ALL USING (
    district_id IN (
      SELECT district_id FROM users
      WHERE users.id = auth.uid()
    )
  );

-- Apply similar policies to other tables
-- (Repeat pattern for vendors, negotiations, savings_realized, etc.)

-- =====
-- DATABASE FUNCTIONS for Calculations
-- =====

-- Calculate total annual spend for a district
CREATE OR REPLACE FUNCTION calculate_annual_spend(p_district_id UUID)
RETURNS DECIMAL AS $$
  SELECT COALESCE(SUM(annual_value), 0)
  FROM contracts
  WHERE district_id = p_district_id
  AND status = 'active';
$$ LANGUAGE SQL STABLE;

-- Calculate YTD savings for a district
CREATE OR REPLACE FUNCTION calculate_ytd_savings(p_district_id UUID)
RETURNS DECIMAL AS $$
  SELECT COALESCE(SUM(savings_amount), 0)
  FROM savings_realized
  WHERE district_id = p_district_id
  AND realized_date >= DATE_TRUNC('year', CURRENT_DATE);
$$ LANGUAGE SQL STABLE;

-- Get upcoming renewals (next N days)
CREATE OR REPLACE FUNCTION upcoming_renewals(p_district_id UUID, p_days INTEGER)
RETURNS TABLE (

```

```

contract_id UUID,
contract_name TEXT,
vendor_name TEXT,
renewal_date DATE,
annual_value DECIMAL,
days_until_renewal INTEGER
) AS $$
SELECT
    c.id,
    c.contract_name,
    v.vendor_name,
    c.renewal_date,
    c.annual_value,
    (c.renewal_date - CURRENT_DATE) AS days_until_renewal
FROM contracts c
LEFT JOIN vendors v ON c.vendor_id = v.id
WHERE c.district_id = p_district_id
AND c.status = 'active'
AND c.renewal_date IS NOT NULL
AND c.renewal_date BETWEEN CURRENT_DATE AND (CURRENT_DATE + p_days)
ORDER BY c.renewal_date ASC;
$$ LANGUAGE SQL STABLE;

```

```

-- =====
-- TRIGGERS for Updated Timestamps
-- =====

```

```

CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER update_districts_updated_at
BEFORE UPDATE ON districts
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

```

CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

```

CREATE TRIGGER update_vendors_updated_at
BEFORE UPDATE ON vendors
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

```

CREATE TRIGGER update_contracts_updated_at

```

```

BEFORE UPDATE ON contracts
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_negotiations_updated_at
BEFORE UPDATE ON negotiations
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

---

## 4. Frontend Implementation (WeWeb)

### Dashboard Structure

**Executive Overview (District View) Key Metrics Cards:** - Total Annual Vendor Spend (large number with trend) - Active Negotiations Pipeline (count with value) - YTD Savings Realized vs Target (progress bar) - Upcoming Renewals (next 90 days count)

**Visualizations:** - Spend by Category (interactive pie/donut chart) - Spending Trends Over Time (line chart) - Top 10 Vendors by Spend (horizontal bar chart) - Savings Funnel (pipeline visualization)

**Alert Cards:** - Contracts expiring in 30 days (urgent renewal attention) - High-value negotiations in progress - Benchmark alerts (spending above peer average)

**Contract Management Features:** - Searchable/filterable contract table - Column sorting (by date, value, status, vendor) - Bulk upload interface with drag-drop - Individual contract detail panels - Document viewer with AI-parsed highlights - Edit/approval workflow interface - Status management (active/expired/under negotiation)

**Table Columns:** - Contract Name - Vendor - Start/End Date - Annual Value - Renewal Date - Status - Actions (View/Edit/Download)

**Savings Tracker Features:** - Negotiation pipeline (drag-drop Kanban stages) - Savings funnel visualization - Success fee calculator (15% savings, 30% donations) - Benchmark comparison charts - ROI dashboard (savings vs Fortify fees)

**Pipeline Stages:** 1. Identified 2. In Progress 3. Vendor Contacted 4. Proposal Received 5. Approved 6. Completed

**Analytics & Insights Reports:** - Spend trends over time (monthly/quarterly/annual) - Vendor consolidation opportunities - Contract renewal calendar view - Utilization metrics (if available) - Peer benchmark comparisons (anonymized)

**Benchmark Intelligence:** - “Your Ed Services spend is 23% above peer districts” - “Similar-sized districts spend \$X less on this vendor” - “Opportunity: \$125k potential savings identified”

**Settings & Admin User Management:** - Add/remove district users - Role assignment (admin/editor/viewer) - Permission management

**Notification Preferences:** - Email alerts for renewals - Savings milestones - Weekly digest options

**Data Management:** - Export to CSV/Excel - Audit log viewer - Document storage management

## WeWeb Component Architecture

```
fortify-platform/  
  pages/  
    login.vue          # Authentication page  
    dashboard.vue      # Executive overview  
    contracts/  
      index.vue        # Contract list/table  
      [id].vue         # Contract detail view  
      upload.vue       # Bulk upload interface  
      review.vue       # AI parsing review  
    negotiations/  
      index.vue        # Pipeline Kanban  
      [id].vue         # Negotiation detail  
    savings/  
      index.vue        # Savings tracker  
      reports.vue      # ROI reports  
    analytics/  
      index.vue        # Analytics dashboard  
    settings/  
      users.vue        # User management  
      preferences.vue  # Settings  
  components/  
    MetricCard.vue     # Reusable metric display  
    ContractTable.vue  # Data table component  
    NegotiationKanban.vue # Drag-drop pipeline  
    SavingsChart.vue   # Chart components  
    UpcomingRenewals.vue # Alert widget  
    SpendByCategory.vue # Pie chart  
    BenchmarkComparison.vue # Peer comparison  
    DocumentViewer.vue # PDF viewer with highlights  
  composables/  
    useAuth.js         # Authentication logic  
    useContracts.js    # Contract CRUD operations  
    useNegotiations.js # Negotiation management  
    useSavings.js      # Savings calculations  
    useAnalytics.js    # Analytics queries  
  supabase/  
    client.js          # Supabase configuration  
  assets/  
    styles/  
      global.css      # Global styles  
    images/  
      logos/          # District logos
```

## Example WeWeb Component: MetricCard.vue

```
<template>
  <div class="metric-card" @click="handleClick">
    <div class="metric-header">
      <h3 class="metric-title">{{ title }}</h3>
      <span class="metric-trend" :class="trendClass">
        <icon :name="trendIcon" />
        {{ changeFormatted }}
      </span>
    </div>

    <div class="metric-value">
      {{ valueFormatted }}
    </div>

    <div class="metric-sparkline" v-if="showSparkline">
      <sparkline :data="sparklineData" />
    </div>

    <div class="metric-subtitle" v-if="subtitle">
      {{ subtitle }}
    </div>
  </div>
</template>

<script setup>
import { computed } from 'vue';

const props = defineProps({
  title: String,
  value: [Number, String],
  change: Number,
  trend: String, // 'up' | 'down' | 'neutral'
  sparklineData: Array,
  subtitle: String,
  showSparkline: {
    type: Boolean,
    default: true
  },
  clickable: {
    type: Boolean,
    default: true
  }
});

const emit = defineEmits(['click']);
```

```

const valueFormatted = computed(() => {
  if (typeof props.value === 'number') {
    return new Intl.NumberFormat('en-US', {
      style: 'currency',
      currency: 'USD',
      minimumFractionDigits: 0,
      maximumFractionDigits: 0
    }).format(props.value);
  }
  return props.value;
});

const changeFormatted = computed(() => {
  if (!props.change) return '';
  const sign = props.change > 0 ? '+' : '';
  return `${sign}${props.change.toFixed(1)}%`;
});

const trendClass = computed(() => ({
  'trend-positive': props.trend === 'up',
  'trend-negative': props.trend === 'down',
  'trend-neutral': props.trend === 'neutral'
}));

const trendIcon = computed(() => {
  switch (props.trend) {
    case 'up': return 'trending-up';
    case 'down': return 'trending-down';
    default: return 'trending-neutral';
  }
});

const handleClick = () => {
  if (props.clickable) {
    emit('click');
  }
};
</script>

<style scoped>
.metric-card {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 16px;
  padding: 24px;
  cursor: pointer;
  transition: all 0.3s ease;

```

```

}

.metric-card:hover {
  background: rgba(255, 255, 255, 0.08);
  border-color: rgba(255, 255, 255, 0.2);
  transform: translateY(-2px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.15);
}

.metric-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 16px;
}

.metric-title {
  font-size: 14px;
  font-weight: 500;
  color: rgba(255, 255, 255, 0.7);
  margin: 0;
}

.metric-trend {
  display: flex;
  align-items: center;
  gap: 4px;
  font-size: 12px;
  font-weight: 600;
  padding: 4px 8px;
  border-radius: 8px;
}

.trend-positive {
  color: #10b981;
  background: rgba(16, 185, 129, 0.1);
}

.trend-negative {
  color: #ef4444;
  background: rgba(239, 68, 68, 0.1);
}

.trend-neutral {
  color: #6b7280;
  background: rgba(107, 114, 128, 0.1);
}

```

```

.metric-value {
  font-size: 36px;
  font-weight: 700;
  color: #ffffff;
  margin-bottom: 12px;
  line-height: 1;
}

.metric-sparkline {
  height: 40px;
  margin-bottom: 8px;
}

.metric-subtitle {
  font-size: 12px;
  color: rgba(255, 255, 255, 0.5);
}
</style>

```

---

## 5. AI Contract Processing Pipeline

### Architecture Overview

```

Contract Upload → Webhook Trigger
      ↓
PDF Text Extraction (PDF.co or Adobe API)
      ↓
Claude API Processing (Structured Output)
      ↓
JSON Parsing & Validation
      ↓
Staging Table in Supabase
      ↓
Human Review Interface (WeWeb)
      ↓
Approval → Main Contracts Table
      ↓
Activity Log & Notifications

```

### Make.com Workflow Configuration

#### Modules in Sequence:

1. **Webhook Trigger**
  - Listen for contract upload events
  - Receive: district\_id, file\_url, uploaded\_by
2. **Supabase Query**
  - Get district context (enrollment, previous enrollment)

- Get uploaded file details
- 3. PDF Text Extraction**
    - Service: PDF.co or Adobe PDF Services
    - Extract raw text from contract PDF
    - Handle multi-page documents
  - 4. HTTP Request to Claude API**
    - Model: claude-sonnet-4-20250514
    - Max tokens: 4000
    - System prompt with extraction instructions
  - 5. Parse JSON Response**
    - Validate JSON structure
    - Handle parsing errors
    - Calculate confidence scores
  - 6. Insert to Staging Table**
    - Supabase insert operation
    - Status: 'pending\_review'
    - Include AI confidence score
  - 7. Error Handling**
    - Retry logic (3 attempts)
    - Error logging to Supabase
    - Notification to admin on failure
  - 8. Success Notification**
    - Slack message to Fortify team
    - Email to district contact
    - In-app notification

## Claude API Prompt Template

```
{
  "model": "claude-sonnet-4-20250514",
  "max_tokens": 4000,
  "system": "You are a contract analysis expert specializing in K-12 school district vendor contracts.",
  "messages": [{
    "role": "user",
    "content": "Analyze this vendor contract and extract key information."
  }]
```

### DISTRICT CONTEXT:

```
- District: ${districtName}
- Current enrollment: ${enrollmentCurrent}
- Previous year enrollment: ${enrollmentPrevious}
- Enrollment change: ${enrollmentChange}%
```

### CONTRACT TEXT:

```
${contractText}
```

### INSTRUCTIONS:

1. Extract all contract details with high accuracy
2. Calculate cost per student metrics

3. Identify negotiation opportunities based on enrollment decline
4. Flag any unusual terms or clauses
5. Assign confidence scores (0-100) for each extracted field

Return ONLY valid JSON with this exact structure (no markdown, no explanation):

```
{
  "vendor_name": "",
  "contract_type": "software|services|supplies|transportation|food_service|other",
  "start_date": "YYYY-MM-DD",
  "end_date": "YYYY-MM-DD",
  "annual_value": number,
  "payment_terms": "",
  "renewal_terms": "",
  "auto_renew": boolean,
  "renewal_notice_days": number,
  "cancellation_terms": "",
  "line_items": [
    {
      "description": "",
      "unit_cost": number,
      "quantity": number,
      "annual_cost": number,
      "category": ""
    }
  ],
  "key_terms": [
    "term description"
  ],
  "negotiation_opportunities": [
    {
      "opportunity": "",
      "reasoning": "",
      "potential_savings_estimate": number
    }
  ],
  "red_flags": [
    "concerning term or clause"
  ],
  "cost_per_student": number,
  "previous_cost_per_student": number,
  "confidence_scores": {
    "vendor_name": 0-100,
    "dates": 0-100,
    "annual_value": 0-100,
    "line_items": 0-100,
    "overall": 0-100
  },
}
```

```
"extraction_notes": ""
}~
}]
}
```

## Human Review Interface (WeWeb)

### Layout: Side-by-Side Comparison

Contract Review Queue

[Filters ]

Original PDF

AI-Extracted Data

[PDF Viewer]

Vendor: ABC Software Inc  
Confidence: 92%

Highlighted  
sections linked  
to extracted  
fields

Start: 2024-01-01  
End: 2026-12-31  
Annual Value: \$125,000

[Page 1 of 12]

[Edit Fields]

Line Items (3):

- Software licenses
- Implementation
  - Annual support

Approve

Reject

Flag for Review

Opportunities:

- Enrollment decline  
leverage: ~\$18k

**Confidence Score Indicators:** - Green (>90%): High confidence, quick approve - Yellow (70-90%): Review recommended - Red (<70%): Manual review required

**Bulk Actions:** - Approve all high-confidence (>90%) - Send batch to manual review - Reject and reprocess

**Learning Loop:** - Track correction patterns - Update Claude prompts based on common errors - Improve extraction accuracy over time

## 6. Authentication & Security

### Supabase Auth Implementation

#### User Signup Flow:

```
// Invite-based signup (districts create accounts via invitation)
const { user, session, error } = await supabase.auth.signUp({
  email: email,
  password: password,
  options: {
    data: {
      district_id: invitationDistrictId,
      role: 'district_viewer', // Default role
      invited_by: adminUserId,
      full_name: fullName
    },
    emailRedirectTo: `https://${districtDomain}.fortifyplatform.com/auth/callback`
  }
})
```

#### Login Flow:

```
// Email/password login
const { data, error } = await supabase.auth.signInWithPassword({
  email: email,
  password: password
})

// After login, retrieve user's district and role
const { data: userData } = await supabase
  .from('users')
  .select('district_id, role, permissions_json')
  .eq('id', data.user.id)
  .single()

// Store in app state for role-based UI rendering
```

#### Row-Level Security (RLS) Enforcement:

```
-- All queries automatically filtered by user's district_id
-- Users cannot access other districts' data even with direct API calls

-- Example: Getting contracts
SELECT * FROM contracts WHERE district_id = '<user_district_id>';
-- RLS policy ensures this is automatically enforced

-- Super admins bypass district isolation
-- Checked via: users.role = 'super_admin'
```

## Role-Based Access Control (RBAC)

### Role Hierarchy:

1. **Super Admin** (Fortify Team)
  - See all districts
  - Manage platform settings
  - Override any permissions
  - Access admin dashboard
2. **District Admin**
  - Full access to their district
  - Manage district users
  - Upload contracts
  - Edit data
  - Configure district settings
3. **District Editor**
  - Upload contracts
  - Edit contract data
  - Create negotiations
  - View all reports
4. **District Viewer**
  - Read-only access
  - View dashboards
  - Export reports
  - Cannot edit or upload

### Permission Checks in WeWeb:

```
// composables/useAuth.js
```

```
export const useAuth = () => {
  const user = ref(null)
  const district = ref(null)
  const role = ref(null)

  const canEdit = computed(() => {
    return ['super_admin', 'district_admin', 'district_editor'].includes(role.value)
  })

  const canManageUsers = computed(() => {
    return ['super_admin', 'district_admin'].includes(role.value)
  })

  const canViewAllDistricts = computed(() => {
    return role.value === 'super_admin'
  })

  return {
    user,
```

```

    district,
    role,
    canEdit,
    canManageUsers,
    canViewAllDistricts
  }
}

```

## Security Best Practices

### 1. Environment Variables (Never Hardcode)

```

SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_ANON_KEY=eyJhbGc...
SUPABASE_SERVICE_KEY=eyJhbGc... (server-side only)
CLAUDE_API_KEY=sk-ant-... (server-side only)
MAKE_WEBHOOK_SECRET=xxx

```

**2. API Key Protection** - Supabase anon key: Safe for client-side (RLS protects data) - Service key: Server-side only (Make.com, Supabase Functions) - Claude API key: Never exposed to client

**3. HTTPS Only** - Enforce SSL/TLS on all connections - HSTS headers enabled - Secure cookie flags

**4. Audit Logging** - All data modifications logged to activity\_log table - Immutable logs (no deletion/editing) - Track: who, what, when, where

**5. Data Encryption** - Database encryption at rest (Supabase default) - Encrypted backups - Document storage encryption (Supabase Storage)

**6. Rate Limiting** - Supabase built-in rate limiting - Additional limits via Supabase Edge Functions if needed

---

## 7. Automation Features

### Built-in Automations (Supabase Functions + Make.com)

#### 1. Renewal Alert System **Trigger:** Daily cron job (12:00 AM UTC)

**Logic:**

*-- Find contracts expiring in 90, 60, 30 days*

```

SELECT
  c.id,
  c.contract_name,
  c.renewal_date,
  c.annual_value,
  d.name as district_name,
  u.email as district_admin_email
FROM contracts c
JOIN districts d ON c.district_id = d.id

```

```
JOIN users u ON u.district_id = d.id AND u.role = 'district_admin'
WHERE c.status = 'active'
AND c.renewal_date IN (
    CURRENT_DATE + INTERVAL '90 days',
    CURRENT_DATE + INTERVAL '60 days',
    CURRENT_DATE + INTERVAL '30 days'
)
```

**Actions:** - Send email notification to district admin - Create in-app notification - Update contract status to 'pending\_renewal' - Log activity

## 2. Negotiation Reminders **Trigger:** Weekly (Monday 9:00 AM)

**Logic:** - Find negotiations in 'in\_progress' status - Filter by: next\_action\_date < TODAY + 7 days - Group by assigned user

**Actions:** - Send weekly digest email to assigned users - List overdue negotiations - Highlight high-priority items - Include next action reminders

## 3. Savings Milestone Notifications **Trigger:** Real-time (when savings record created)

**Milestones:** - First \$50k saved - \$100k total savings - \$250k total savings - 4x ROI achieved (target met)

**Actions:** - Celebratory email to district leadership - In-app banner notification - Update dashboard with achievement badge - Notify Fortify team (potential case study)

## 4. Stale Contract Flagging **Trigger:** Monthly (1st of month)

**Logic:**

```
-- Find contracts over 5 years old that haven't been reviewed
SELECT * FROM contracts
WHERE status = 'active'
AND start_date < CURRENT_DATE - INTERVAL '5 years'
AND reviewed_at IS NULL
OR reviewed_at < CURRENT_DATE - INTERVAL '1 year'
```

**Actions:** - Flag as 'needs\_review' - Add to review queue - Notify district admin

## 5. Monthly Executive Report **Trigger:** Monthly (Last day of month, 8:00 PM)

**Report Contents:** - Total spend this month - YTD savings - Active negotiations pipeline - Contracts renewed this month - Upcoming renewals (next 90 days) - Key metrics vs. last month - Benchmark comparison

**Actions:** - Generate PDF report - Email to superintendent + district admin - Store in document library

## 6. Vendor Performance Tracking **Trigger:** Quarterly

**Logic:** - Analyze contract renewals vs. cancellations - Track savings achieved per vendor - Flag vendors with multiple issues

**Actions:** - Generate vendor scorecard - Identify consolidation opportunities - Recommend vendors for renegotiation

## **Make.com Automation Recipes**

### **Recipe 1: Contract Upload → AI Processing**

Webhook (Contract Uploaded)  
↓  
Get District Context (Supabase)  
↓  
Extract PDF Text (PDF.co)  
↓  
Process with Claude API  
↓  
Insert to Staging (Supabase)  
↓  
Notify Team (Slack + Email)

### **Recipe 2: Savings Approved → Success Fee Invoice**

Webhook (Savings Approved)  
↓  
Calculate Success Fee (15% or 30%)  
↓  
Update Google Sheet (Invoice Tracker)  
↓  
Create Invoice (QuickBooks API)  
↓  
Send Email (District Admin)  
↓  
Log Activity (Supabase)

### **Recipe 3: New District Onboarded → Setup Workflow**

Webhook (New District Created)  
↓  
Create Database Tenant (Supabase)  
↓  
Generate Welcome Email  
↓  
Create Admin User Account  
↓  
Send Onboarding Guide  
↓  
Schedule Kickoff Call (Calendly API)  
↓  
Notify Fortify Team (Slack)

## Recipe 4: Benchmark Data Update

Schedule (Weekly)

↓

Query All Districts (Supabase)

↓

Calculate Peer Benchmarks

↓

Update Benchmark Table

↓

Flag Outliers (>20% above average)

↓

Send Opportunity Alerts

---

## 8. MVP Timeline with Claude Code

### 4-5 Week Rapid Development Plan

#### Week 1: Foundation (You + Claude Code) Day 1: Database Setup

claude-code "Create complete Supabase schema for multi-tenant school district vendor management platform. Include all tables, RLS policies, indexes, and database functions."

- Review generated schema
- Deploy to Supabase dev environment
- Test RLS policies with sample data

#### Day 2: Authentication Flow

claude-code "Implement Supabase Auth with district-level isolation. Include signup, login, password reset, and role-based access."

- Configure auth settings
- Test user flows
- Set up test districts and users

#### Day 3: WeWeb Project Setup

claude-code "Initialize WeWeb project structure with pages for dashboard, contracts, negotiations, savings. Include Supabase client configuration."

- Create project in WeWeb
- Configure Supabase connection
- Set up routing and navigation

#### Day 4: Basic Dashboard

claude-code "Create executive dashboard with metric cards for total spend, active negotiations, YTD savings, and upcoming renewals. Connect to Supabase queries."

- Test real-time data updates
- Responsive design verification

### Day 5: Contract Table

claude-code "Build contract management table with search, filters, sorting, and pagination. Include CRUD operations."

- Test with sample contracts
- Verify RLS enforcement

**Weekend:** Buffer time for adjustments

---

## Week 2: Core Features Day 1: Contract Upload UI

claude-code "Create contract upload interface with drag-drop, file validation, progress indicators, and preview functionality."

- Connect to Supabase Storage
- Test multi-file uploads
- Implement error handling

### Day 2: AI Parsing Workflow

claude-code "Build Make.com scenario for contract parsing: webhook → PDF extraction → Claude API → staging table → notifications."

- Configure PDF.co integration
- Set up Claude API calls
- Test with real contracts

### Day 3: Review Interface

claude-code "Create staging/review interface with side-by-side PDF viewer and extracted data. Include confidence scoring and bulk approval."

- Test accuracy of extractions
- Implement correction tracking
- Build approval workflow

### Day 4: Metrics Calculations

claude-code "Implement real-time calculations for total spend, savings, cost per student, and ROI metrics. Create Supabase functions."

- Verify calculation accuracy
- Test performance with large datasets

**Day 5: Testing & Bug Fixes** - End-to-end testing - Fix critical issues - Performance optimization

---

## Week 3: Advanced Features Day 1: Negotiations Pipeline

claude-code "Build Kanban-style negotiation pipeline with drag-drop stages, status updates, and assignment functionality."

- Test state management
- Implement real-time updates

## Day 2: Savings Tracker

claude-code "Create savings tracker with charts, ROI calculations, success fee computation, and milestone tracking."

- Integrate chart library
- Test calculations

## Day 3: Analytics Dashboard

claude-code "Build analytics page with spending trends, vendor comparisons, benchmark intelligence, and exportable reports."

- Create data visualizations
- Implement export functionality

## Day 4: Renewal Alerts

claude-code "Set up automated renewal alerts using Supabase cron and Make.com. Include 90/60/30 day notifications."

- Configure email templates
- Test notification delivery

## Day 5: Settings & Admin

claude-code "Create user management, notification preferences, and district settings pages."

- Test permission enforcement
- Implement audit logging

---

**Week 4: Pilot Preparation Day 1: Orange USD Onboarding** - Create Orange USD district account - Set up admin users (Sulema, Connie) - Configure district branding - Load initial data

## Day 2: Custom Branding

claude-code "Implement white-label branding: custom domain setup, district logo integration, color scheme customization."

- Test subdomain routing
- Verify branded experience

**Day 3: Data Migration** - Import existing Orange USD contracts - Run AI parsing on historical data - Verify accuracy and completeness

## Day 4: Training & Documentation

claude-code "Generate user documentation including quick start guide, video script for tutorials, and admin handbook."

- Create video tutorials (Loom)
- Build help center content

**Day 5: Final Testing** - User acceptance testing - Performance testing - Security audit - Bug fixes

---

**Week 5: Pilot Launch & Iteration** **Week 5-7: Live with Orange USD** - Daily monitoring - User feedback collection - Rapid bug fixes - Feature refinements - Monthly check-ins

**Feedback Loop:** - Weekly survey: What's working? What's not? - Usage analytics: Which features are used most? - Support tickets: Common pain points? - Feature requests: What do they want next?

**Iteration Priorities:** 1. Critical bugs (< 24 hours) 2. Usability issues (< 1 week) 3. Feature enhancements (backlog)

---

## Claude Code Efficiency Gains

**Traditional Development:** - 12+ weeks for MVP - \$50-100k in developer costs - Risk of technical debt - Limited iteration speed

**With Claude Code:** - 4-5 weeks for MVP - ~\$100 in API credits - Clean, documented code - Rapid iteration capability

**Time Savings Breakdown:**

| Task                 | Traditional      | Claude Code      | Savings    |
|----------------------|------------------|------------------|------------|
| Database schema      | 1 week           | 1 day            | 80%        |
| Auth implementation  | 1 week           | 1 day            | 80%        |
| Dashboard components | 2 weeks          | 3 days           | 79%        |
| AI integration       | 2 weeks          | 2 days           | 86%        |
| Testing & docs       | 1 week           | 2 days           | 71%        |
| <b>Total</b>         | <b>12+ weeks</b> | <b>4-5 weeks</b> | <b>67%</b> |

---

## 9. Cost Analysis

### Monthly Operating Costs

**MVP Phase (1-5 Districts) Infrastructure:** - Supabase Pro: \$25/month - WeWeb Scale: \$199/month (unlimited users) - Make.com Pro: \$29/month - Claude API: ~\$50-100/month (usage-based) - PDF.co API: \$29/month - Domain & hosting: \$20/month - **Subtotal: ~\$352-402/month**

**Tools & Services:** - Email service (SendGrid): \$15/month - Monitoring (Sentry): \$26/month - Analytics (PostHog): \$0 (free tier) - **Subtotal: ~\$41/month**

**Total Monthly: ~\$393-443**

---

**Scale Phase (10-20 Districts) Infrastructure:** - Supabase Team: \$99/month - WeWeb Scale: \$199/month - Make.com: \$99/month (higher ops volume) - Claude API: ~\$300-500/month - PDF.co: \$99/month (higher volume) - CDN & hosting: \$50/month - **Subtotal: ~\$846-1,046/month**

**Tools & Services:** - Email service: \$49/month - Monitoring: \$50/month - Analytics: \$29/month - Customer support (Intercom): \$74/month - **Subtotal: ~\$202/month**

**Total Monthly: ~\$1,048-1,248**

---

**Enterprise Phase (50+ Districts) Infrastructure:** - Supabase Enterprise: \$599/month - WeWeb Scale: \$199/month - Make.com: \$299/month (enterprise ops) - Claude API: ~\$1,000-2,000/month - PDF.co Enterprise: \$199/month - CDN & hosting: \$200/month - **Subtotal: ~\$2,496-3,496/month**

**Tools & Services:** - Email service: \$99/month - Monitoring: \$99/month - Analytics: \$99/month - Support tools: \$150/month - **Subtotal: ~\$447/month**

**Total Monthly: ~\$2,943-3,943**

---

## Revenue Model

### Pricing Tiers:

**Pilot Program** - Price: \$25,000/year - Services: Full platform access, manual contract analysis - Target: 5-10 districts - Annual Revenue: \$125k-250k

**Standard Plan** - Price: \$75,000/year - Services: Platform + quarterly reviews + benchmarking - Target: 10-30 districts - Annual Revenue: \$750k-2.25M

**Premium Plan** - Price: \$125,000/year - Services: Platform + monthly reviews + dedicated support + custom reports - Target: 20-50 districts - Annual Revenue: \$2.5M-6.25M

**Success Fees (Additional)** - 15% of realized savings - 30% of secured donations - Average per district: \$15-50k annually

---

## Profitability Analysis

**Year 1 Scenario (Conservative) Clients:** - 5 Pilot districts @ \$25k = \$125k - 10 Standard @ \$75k = \$750k - Total Subscription Revenue: \$875k

**Success Fees:** - Average \$25k per district × 15 districts - Total Success Fee Revenue: \$375k

**Total Revenue: \$1,250,000**

**Costs:** - Infrastructure: \$15k (avg \$1,250/mo) - Your time/salary: \$150k - Marketing: \$50k - Legal/admin: \$25k - Total Costs: \$240k

**Net Profit: \$1,010,000 Margin: 80.8%**

---

**Year 2 Scenario (Growth) Clients:** - 30 Standard @ \$75k = \$2.25M - 10 Premium @ \$125k = \$1.25M - Total Subscription Revenue: \$3.5M

**Success Fees:** - Average \$30k per district  $\times$  40 districts - Total Success Fee Revenue: \$1.2M

**Total Revenue: \$4,700,000**

**Costs:** - Infrastructure: \$40k - Team (3 people): \$400k - Sales/marketing: \$200k - Operations: \$100k - Total Costs: \$740k

**Net Profit: \$3,960,000 Margin: 84.3%**

---

**Year 3 Scenario (Scale) Clients:** - 50 Standard @ \$75k = \$3.75M - 30 Premium @ \$125k = \$3.75M - Total Subscription Revenue: \$7.5M

**Success Fees:** - Average \$35k per district  $\times$  80 districts - Total Success Fee Revenue: \$2.8M

**Total Revenue: \$10,300,000**

**Costs:** - Infrastructure: \$80k - Team (8 people): \$1M - Sales/marketing: \$500k - Operations: \$200k - Total Costs: \$1.78M

**Net Profit: \$8,520,000 Margin: 82.7%**

---

### **Cost Comparison: WeWeb vs Retool vs Custom**

**At 20 Districts with 50 Total Users WeWeb Stack:** - WeWeb: \$199/month - Infrastructure: \$1,000/month - **Total: \$1,199/month = \$14,388/year**

**Retool Stack:** - Retool: 50 users  $\times$  \$50 = \$2,500/month - Infrastructure: \$1,000/month - **Total: \$3,500/month = \$42,000/year - Extra cost: \$27,612/year (192% more expensive)**

**Custom Development:** - Initial build: \$72,000 - Maintenance: \$3,000/month = \$36,000/year - **Year 1 Total: \$108,000 - Ongoing: \$36,000/year**

**WeWeb Savings:** - vs Retool: \$27,612/year - vs Custom (Year 1): \$93,612 - vs Custom (ongoing): \$21,612/year

---

## **10. Implementation Roadmap**

### **Phase 1: MVP (Weeks 1-5)**

**Deliverables:** - Supabase database with full schema - WeWeb frontend with core pages - Authentication & RLS - Contract upload & management - AI parsing with review interface - Executive dashboard - Negotiations pipeline - Savings tracker - Basic automations

**Success Criteria:** - Orange USD successfully onboarded - 90% AI parsing accuracy - Sub-2-second page loads - Zero critical security issues - Positive user feedback

---

## **Phase 2: Pilot Expansion (Months 2-3)**

**Objectives:** - Onboard 3-5 additional pilot districts - Collect extensive user feedback - Iterate on UX/UI - Add requested features - Build case studies

**New Features:** - Advanced filtering and search - Bulk operations - Custom report builder - Mobile app (optional) - Slack/Teams integration

**Metrics to Track:** - User engagement (DAU/MAU) - Feature adoption rates - Time to value (first savings) - NPS score

---

## **Phase 3: Commercial Launch (Months 4-6)**

**Objectives:** - Launch marketing website - Develop sales materials - Implement onboarding automation - Scale infrastructure - Build sales pipeline

**Marketing:** - Website with product demos - Case studies (Orange USD) - Conference presentations (ASBO, CASBO) - Content marketing (blog, webinars) - Partnership with procurement associations

**Sales Process:** - Discovery call - Demo customized to district - Proposal with ROI projection - Contract negotiation - Onboarding (2 weeks)

---

## **Phase 4: Scale & Advanced Features (Months 7-12)**

**New Capabilities:** - Advanced benchmarking (AI-powered) - Predictive analytics (renewal risk) - Vendor marketplace - RFP/bidding module - Integration hub (ERP systems)

**Enterprise Features:** - SSO/SAML - Multi-district/consortium management - Custom branding per district - API access for districts - White-label options

**Team Growth:** - Hire sales rep (Month 6) - Hire customer success manager (Month 8) - Hire additional developer (Month 10) - Contract with fractional CFO (Month 6)

---

## **Phase 5: Market Domination (Year 2+)**

**Expansion:** - Launch in adjacent markets (charter schools, universities) - Geographic expansion (state-by-state) - Product line extensions (HR, facilities) - Acquisition opportunities

**Technology Evolution:** - Migration to custom stack (if needed) - Advanced ML models - Mobile apps (iOS/Android) - Offline capabilities

**Exit Strategy Options:** - Bootstrap to profitability (primary) - Strategic acquisition (EdTech company) - Private equity rollout - Merge with complementary SaaS

---

## 11. Next Steps

### Immediate Actions (This Week)

**Day 1-2: Technical Foundation** 1. Set up Supabase project - Create organization account - Initialize database - Configure environments (dev/staging/prod)

2. Generate database schema with Claude Code

```
claude-code "Create complete Supabase schema [full prompt]"
```

3. Deploy schema to dev environment

4. Test RLS policies with sample data

**Day 3-4: WeWeb Setup** 1. Create WeWeb account and project 2. Connect to Supabase 3. Build first dashboard page with Claude Code `bash` `claude-code "Create executive dashboard [full prompt]"`

4. Test authentication flow

5. Deploy to staging environment

**Day 5: AI Pipeline Prototype** 1. Set up Make.com account 2. Create webhook endpoint 3. Test PDF extraction (PDF.co) 4. Test Claude API integration 5. End-to-end test with sample contract

---

### Week 2 Goals

**Technical:** - Complete all core database tables - Build contract management interface - Implement upload functionality - Create review/staging interface

**Business:** - Finalize pricing model - Draft sales materials - Create onboarding checklist - Schedule Orange USD kickoff

---

### Key Decisions Required

**1. Branding Decision** - Platform name: ProClear, SchoolSpend, ProIntel, or Procision? - Logo and color scheme - Custom domain setup

**2. Pilot Pricing** - Orange USD contract terms - Success fee structure (15% vs 30%) - Payment schedule

**3. Feature Prioritization** - Which features for MVP vs Phase 2? - Mobile app priority? - Integration priorities?

---

### Resources & Tools Setup

**Required Accounts:** - Supabase (database & auth) - WeWeb (frontend) - Make.com (automation) - Anthropic (Claude API) - PDF.co (PDF extraction) - SendGrid (email) - Sentry (monitoring) - Loom (video tutorials)

**Development Tools:** - Claude Code (for rapid development) - VS Code (code editing) - Postman (API testing) - Figma (design reference, optional)

---

## Success Metrics (90 Days)

**Technical:** - Platform uptime: >99.5% - Page load time: <2 seconds - AI parsing accuracy: >90%  
- Zero critical security issues

**Business:** - Districts onboarded: 5-10 - Contracts processed: 100+ - Total savings identified: \$500k+ - User satisfaction (NPS): >50

**Financial:** - MRR: \$10-20k - Customer acquisition cost: <\$5k - Lifetime value: >\$100k - Burn rate: <\$5k/month

---

## Conclusion

### Why This Stack Wins

**WeWeb + Supabase + Claude Code** provides the optimal balance of:

**Speed to Market:** 4-5 weeks vs 12+ weeks **Cost Efficiency:** ~\$400/month vs \$3,500+/month  
**Professional Quality:** Looks like \$500k custom build **Scalability:** From 5 to 500+ districts seamlessly  
**Future-Proof:** Can export code, no vendor lock-in **Margins:** 98%+ gross margins at scale  
**Control:** Full ownership of data and infrastructure

### The Fortify Advantage

This platform positions Fortify Partners to:

- 1. Dominate the K-12 vendor optimization market**
  - First-mover advantage with comprehensive platform
  - Network effects (more districts = better benchmarks)
  - High switching costs once integrated
- 2. Scale efficiently without technical debt**
  - Modern tech stack
  - Automation at core
  - AI-powered from day one
- 3. Maximize profitability**
  - Low operational costs
  - High-value pricing justified by ROI
  - Multiple revenue streams (subscriptions + success fees)
- 4. Expand strategically**
  - Adjacent markets (charters, universities)
  - Product line extensions (HR, facilities)
  - Geographic expansion

### Your Competitive Moat

- 1. Data Network:** More districts = better benchmarks = more value

2. **AI Expertise:** Continuously improving contract analysis
3. **Industry Knowledge:** Deep understanding of K-12 procurement
4. **Trust & Relationships:** Track record with districts like Orange USD
5. **Technology:** Modern, scalable, professional platform

## The Path Forward

**Weeks 1-5:** Build MVP with Orange USD **Months 2-3:** Pilot expansion (5-10 districts) **Months 4-6:** Commercial launch and sales ramp **Months 7-12:** Scale to \$1M+ ARR **Year 2+:** Dominate market, strategic optionality

**This is not just a platform—it's a category-defining business.**

---

## Appendix

### Glossary

**MRR:** Monthly Recurring Revenue **ARR:** Annual Recurring Revenue **NPS:** Net Promoter Score  
**RLS:** Row-Level Security **SaaS:** Software as a Service **MVP:** Minimum Viable Product **ROI:** Return on Investment **DAU/MAU:** Daily Active Users / Monthly Active Users

### Additional Resources

**Documentation:** - Supabase: <https://supabase.com/docs> - WeWeb: <https://docs.weweb.io> - Make.com: <https://www.make.com/en/help> - Claude API: <https://docs.anthropic.com>

**Learning Resources:** - WeWeb University: <https://university.weweb.io> - Supabase YouTube: <https://youtube.com/@supabase> - Make.com Academy: <https://www.make.com/en/academy>

**Community:** - WeWeb Community: <https://community.weweb.io> - Supabase Discord: <https://discord.supabase.com> - Indie Hackers: <https://www.indiehackers.com>

---

**Document Version:** 1.0

**Last Updated:** February 4, 2026

**Author:** Strategic Technology Advisor

**Prepared For:** Fortify Partners - Adam Johnson